

Lab Report 1

Rapid Control Prototyping (RCP) with MATLAB/Simulink

Robotics Lab: Control Systems

IRO-4 SS26

Date: April 9, 2026

Professor: Prof. Martin Löser

Prof. Thomas Glowacz

Group: 7

Pholasith Laoruengrong

Matrikelnr.: 4224003

Harsh Rameshbhai Mangukiya

Matrikelnr.: 4224076

Task 1: Identification of System 3

The identification of System 3 was performed manually using its step response. The response shows no oscillations and can be approximated by a **first-order lag**. The **steady-state gain** K was determined from the ratio of final output to final input: $K = \frac{y_{\infty}}{u_{\infty}}$. The **time constant** T was estimated using the 63% method: measuring the time at which the output reached $0.63 \times y_{\infty}$, starting from the actual step input time $t_0 = 1$ s (since the step occurs at $t = 1$ s). This method demonstrates how a first-order system can be identified from its step response by observing the time constant and steady-state gain.

Identified model: PT1 (First-order system) with transfer function $G_{S3}(s) = \frac{1.4762}{0.48s+1}$

After simulating the suggested transfer function and comparing it with the measured step response, the parameters were slightly adjusted to improve the match between the model output and the experimental data. The corrected transfer function is $G_{S3}(s) = \frac{1.485}{0.485s+1}$

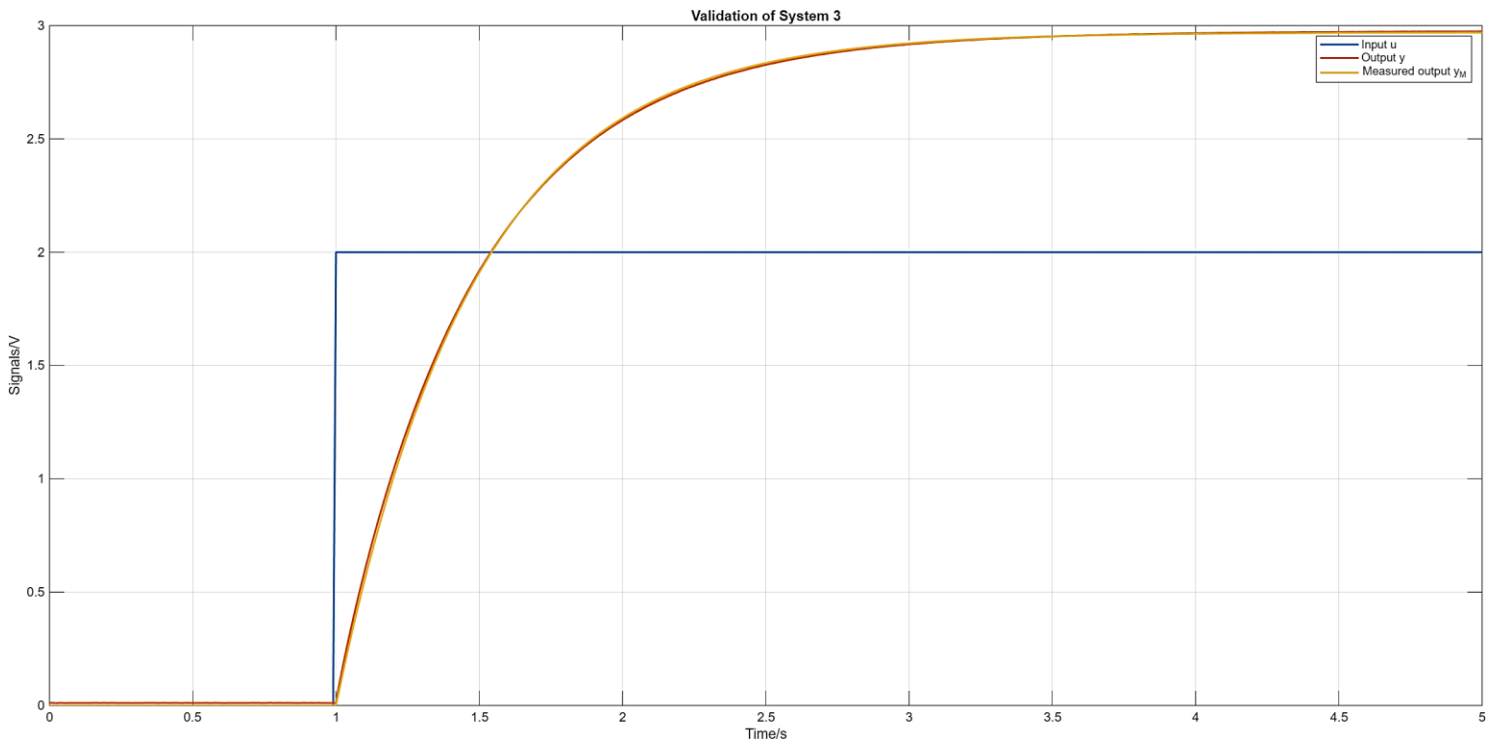


Figure 1: Step response of System 3 and fitted PT1 model

Task 2: Identification of System 2

By observing the step response of System 2, we noticed **oscillatory behavior**, indicating a **second-order dynamic system** (PT2). As in Task 1, the **steady-state gain** K was calculated from the ratio of the final output y_{∞} to the input u_{∞} . The **damping ratio** D was estimated from the overshoot M_p using: $M_p = \frac{y_{\max} - y_{\infty}}{y_{\infty}}$, and the natural frequency Ω was derived from the period of oscillation T between successive peaks: $\Omega = \frac{2\pi}{T}$, $\Omega_0 = \frac{\Omega}{\sqrt{1-D^2}}$

Identified model: PT2 (Second-order system) with transfer function (assuming standard second order model)

$$G_{S2}(s) = \frac{0.7555}{\frac{1}{4.7941^2} s^2 + \frac{2 \cdot 0.4}{4.7941} s + 1}$$

The oscillatory response indicates that the system contains **energy-storing components** (e.g., inductors, capacitors, or equivalent in mechanical systems). The identification method demonstrates how **damping ratio and natural frequency** can be estimated directly from step response characteristics.

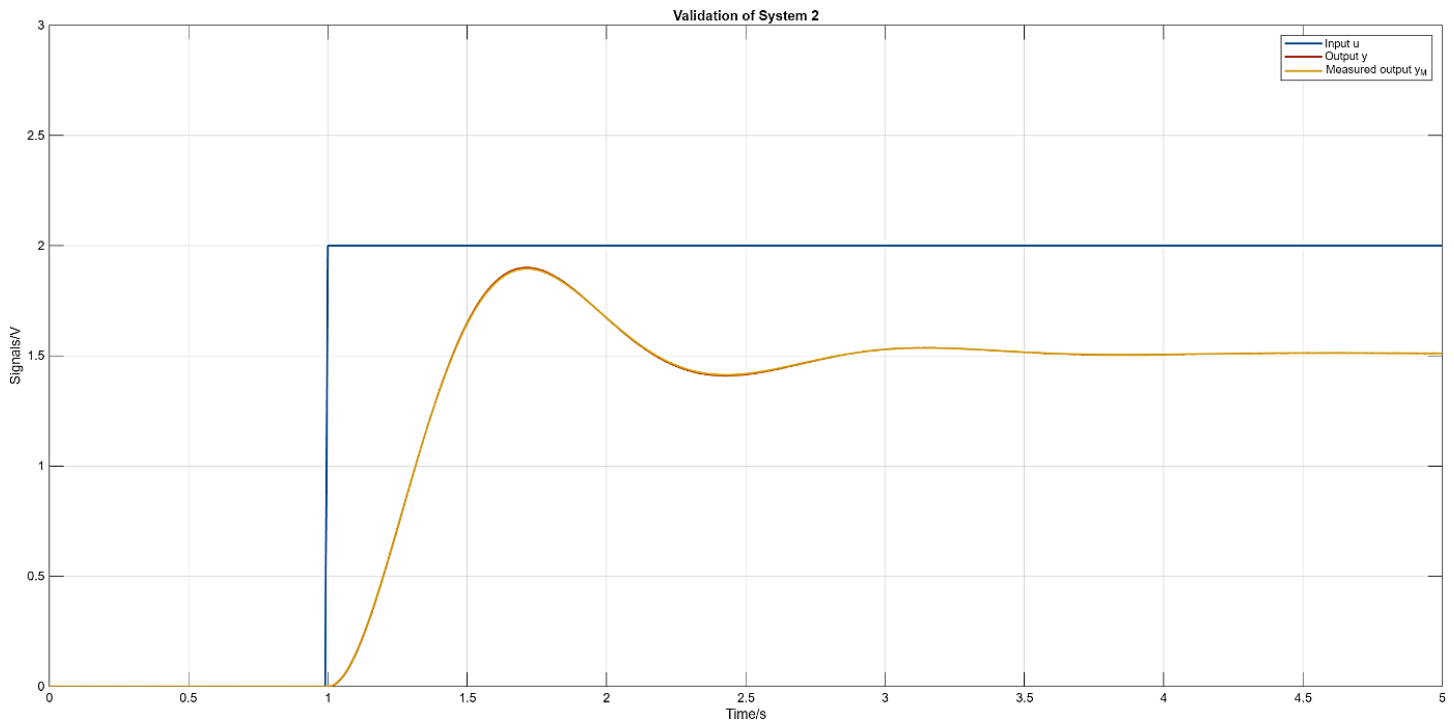


Figure 2: Step response of System 2 and fitted PT2 model

Task 3: Implementation, Test and Tuning of P, PI and PID controllers

The control plant consists of System 3 cascaded with System 2. Instead of using formal design techniques such as root locus or frequency-response methods (Bode/Nyquist), the controllers were tuned manually using a **trial-and-error approach**. During tuning, the following objectives were maintained: Oscillations must **decay exponentially**. The system output must reach a **bounded steady-state value**. **No steady-state error** should remain after a set-point change. The system should reach a new set-point within approximately **2.4 s**, with minimal overshoot and negligible oscillation amplitude.

Controller Tuning Process:

- **P controller:** Increasing the proportional gain K_R reduces the steady-state error but increases oscillations.
- **PI controller:** Adding integral action eliminates steady-state error but may increase overshoot.

- **PID controller:** Adding derivative action counteracts overshoot by anticipating future changes, improving stability and minimizing oscillations.

	K _R	T _I /s	T _D /s
P controller	1.5	-	-
PI controller	0.6	0.5	-
PID controller	0.37	0.5	0.8

Table: Optimized Controller Parameters

Observations:

- Increasing K_R improves response speed but can destabilize the system.
- Integral action removes steady-state error but may induce overshoot.
- Derivative action improves stability and reduces overshoot.

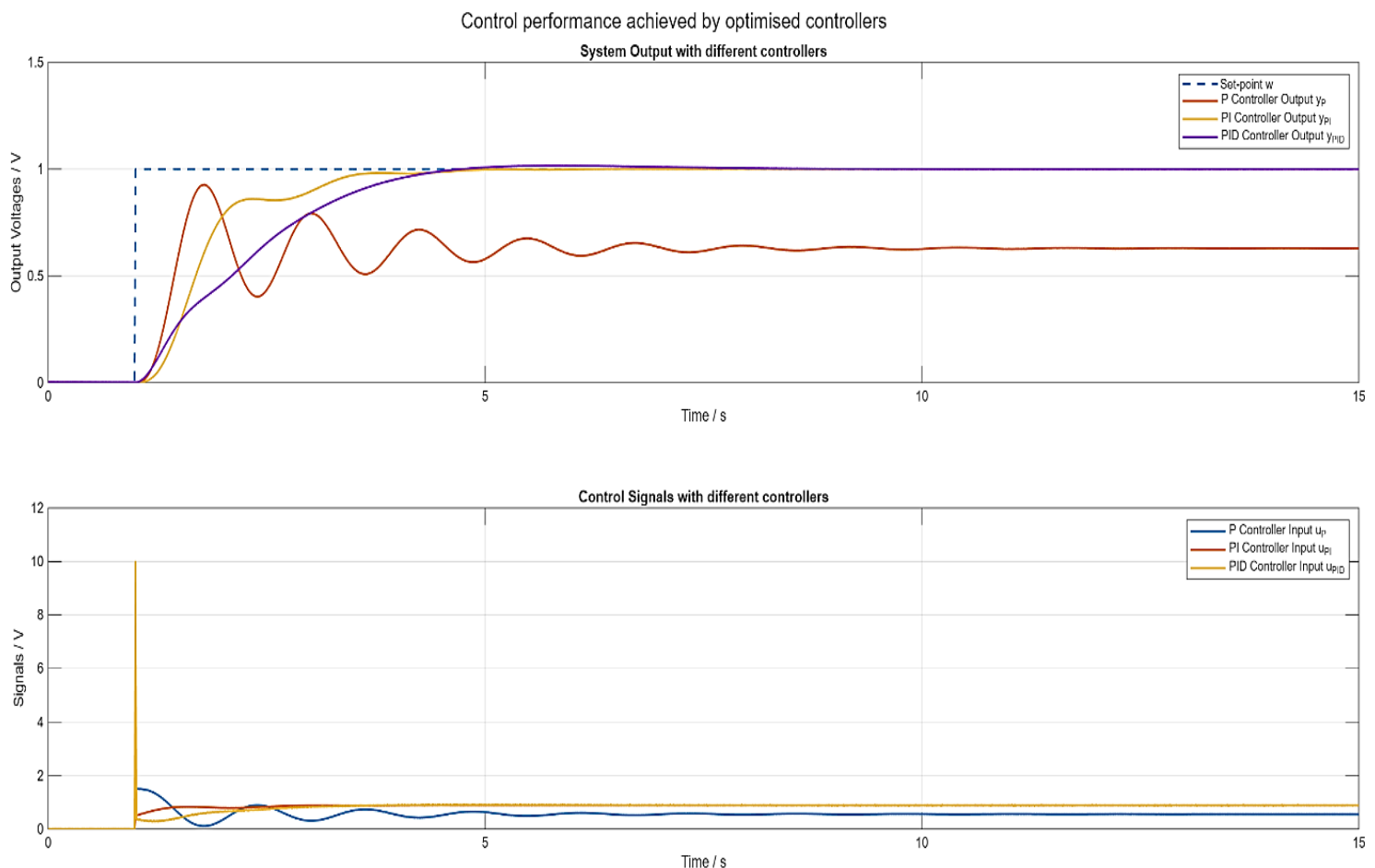


Figure 3: Closed-loop comparison (P vs PI vs PID)

Results:

- **P controller:** steady-state error remains
- **PI controller:** eliminates steady-state error but overshoot increases
- **PID controller:** fastest settling and minimal overshoot